

SIMATS ENGINEERING
ASSESSMENT TOOL 2- COMPARATIVE ANALYSIS

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO4	Assessment:	ASSESSMENT TOOL2- COMPARATIVE ANALYSIS
Weightage:	30%	Total Marks:	25
CO4 Statement:	<i>Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.</i>		
Student Name:	KV GAGAN YADAV	Reg.No.:	192412406
Department:	ECE	Date:	25-08-26

ANNEXURE A
COMPARATIVE ANALYSIS

1. Compare L1, L2, and L3 cache levels in terms of latency, bandwidth, and throughput, and analyze their impact on processor performance.
2. Differentiate SRAM and DRAM by evaluating their latency, bandwidth, and throughput characteristics in cache and main memory design.
3. Compare virtual memory with paging and cache memory mechanisms in terms of access latency and system throughput.
4. Analyze the differences between NAND Flash and DRAM with respect to latency, bandwidth, and suitability for hierarchical memory systems.
5. Compare MRAM and RRAM technologies by examining their performance in terms of access time, throughput, and energy efficiency.

Code of Conduct:

I KV GAGAN YADAV (192412406) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

KV GAGAN YADAV

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Hierarchical Memory System Implementation

A hierarchical memory system combines different memory technologies so that the processor gets frequently used data quickly, while less frequently used data can be stored in larger and cheaper memory. The hierarchy can be implemented as:

CPU Registers → L1 Cache → L2 Cache → L3 Cache → DRAM → NAND Flash

Advanced non-volatile memories such as MRAM and RRAM can be incorporated between or alongside SRAM/DRAM depending on the system design.

1. Memory hierarchy based on performance

Memory level / technology	Typical latency	Bandwidth	Throughput	Primary role
L1 Cache (SRAM)	~1–4 cycles	Very high	Very high	Immediate CPU data/instructions
L2 Cache (SRAM)	~5–15 cycles	High	Very high	Secondary fast cache
L3 Cache (SRAM)	~20–50+ cycles	High	High	Large shared cache
DRAM ~50–100+ ns	Very high	High	Main memory	
MRAM	Tens of ns or potentially lower, technology-dependent			High High Fast non-volatile memory
RRAM	Tens of ns or potentially lower, technology-dependent			High High Dense non-volatile memory
NAND Flash	~tens–hundreds of μ s for SSD-level operations, workload-dependent			High sequential bandwidth
	High sequential throughput, lower random throughput			Persistent storage

The exact values vary significantly with processor generation, cache design, memory controller, DRAM type, SSD architecture, and MRAM/RRAM implementation.

2. Implementing L1, L2 and L3 caches

L1 Cache

L1 should be implemented using high-speed SRAM and placed directly inside or very close to each processor core.

- Small capacity minimizes access time.
- Separate instruction and data caches are commonly used.
- Very low latency allows the CPU pipeline to continue with minimal delay.
- High bandwidth supports the processor's immediate instruction and data requirements.

Implementation principle:

- CPU Core $\rightarrow$ L1 I-Cache / L1 D-Cache

If the required cache line is present, the processor receives it immediately. Otherwise, an L1 miss causes a lookup in L2.

L2 Cache

L2 provides more capacity than L1 while accepting somewhat higher latency.

- L1 Miss $\rightarrow$ L2 Lookup

L2 can be private to a core or shared by a small group of cores. Its purpose is to capture data that does not fit into L1 while still avoiding expensive DRAM accesses.

L3 Cache

L3 is normally much larger and is frequently shared among processor cores.

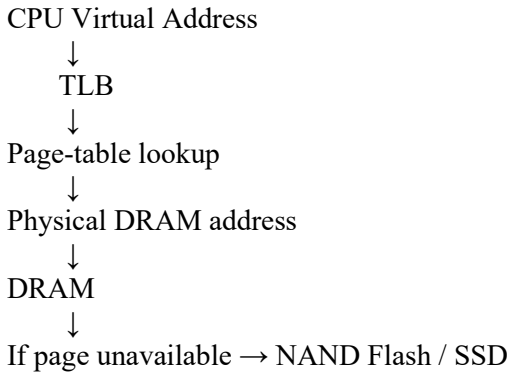
- L2 Miss $\rightarrow$ Shared L3 Lookup

A successful L3 lookup avoids a DRAM access. Although L3 is slower than L1 and L2, its larger capacity can substantially increase the overall cache hit rate.

3. Implementing virtual memory with paging

Virtual memory operates above the physical memory hierarchy and provides each process with a virtual address space.

A simplified implementation is:



The TLB (Translation Lookaside Buffer) caches recently used virtual-to-physical address translations.

TLB hit: translation is obtained quickly.

TLB miss: page tables must be consulted, adding latency.

Page fault: required page is not in DRAM and must be retrieved from secondary storage, producing a very large latency penalty.

Therefore, paging should keep actively used pages in DRAM and move relatively inactive pages to NAND Flash when physical memory is insufficient.

Throughput consideration

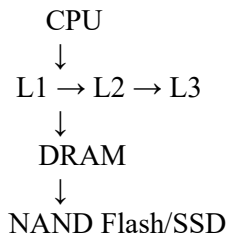
Excessive page faults cause thrashing, in which the system spends much of its time moving pages rather than executing applications. An effective implementation therefore combines:

- TLB caching
- Efficient page replacement
- Sufficient DRAM capacity
- Large sequential transfers where possible
- Working-set management

4. Integrating DRAM and NAND Flash

DRAM should serve as the primary working memory because it offers substantially lower latency and better random-access performance than NAND Flash.

NAND Flash should provide persistent storage:



When an application accesses data already in DRAM, the CPU avoids the extremely high latency associated with storage access. When a required virtual-memory page is absent from DRAM, the operating system retrieves it from NAND Flash.

Thus:

- DRAM: optimize active working-set access.
- NAND Flash: optimize capacity, persistence, and cost per bit.
- Cache: optimize CPU access latency.

5. Integrating MRAM and RRAM

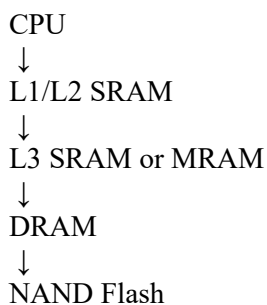
MRAM and RRAM can be introduced as non-volatile intermediate memory technologies.

MRAM

MRAM can be used where both fast access and persistence are valuable. Potential applications include:

- Embedded processor memory
- Persistent cache structures
- Fast non-volatile buffers
- Systems requiring high write endurance

A possible hierarchy is:



MRAM can potentially replace or supplement SRAM in selected applications, especially where eliminating refresh or retaining state during power loss is valuable.

RRAM

RRAM can provide high-density non-volatile storage and is particularly interesting for specialized architectures.

For example:

CPU / Accelerator

↓
SRAM Cache

↓
DRAM

↓
RRAM

↓
NAND Flash

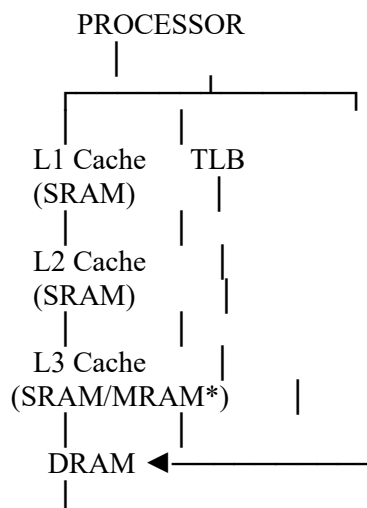
RRAM can also be incorporated into in-memory or neuromorphic computing architectures, where computation is performed close to or within the memory array.

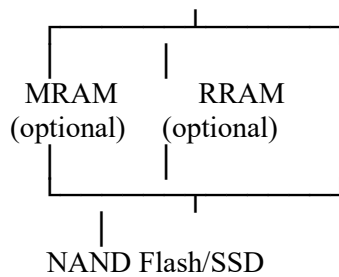
6. Overall implementation strategy

A practical hierarchical system should assign each technology according to its characteristics:

- | Technology | Best optimization target | Position |
|------------|--|---------------------------------|
| SRAM | Minimum latency, maximum cache throughput | L1/L2/L3 |
| DRAM | Capacity + high main-memory bandwidth | Main memory |
| MRAM | Fast, persistent, energy-efficient memory | Intermediate/specialized memory |
| RRAM | High-density, non-volatile and specialized computing | Intermediate/specialized memory |
| NAND Flash | Persistence + high capacity | Secondary storage |

The resulting architecture can therefore be represented as:





* MRAM substitution for L3 is architecture-dependent rather than a universal implementation.

Performance analysis

The hierarchy improves system performance through locality of reference:

- L1 provides the lowest latency and highest immediate throughput.
- L2 catches L1 misses with slightly greater latency.
- L3 catches many L2 misses before they reach DRAM.
- DRAM provides large-capacity, high-bandwidth working memory.
- MRAM/RRAM can add non-volatility and energy advantages where appropriate.
- NAND Flash supplies inexpensive, persistent high-capacity storage.
- Paging manages movement between DRAM and secondary storage, while the TLB reduces address-translation overhead.

Consequently, an effective hierarchical memory system does not attempt to use one memory technology everywhere. Instead, it places low-latency, high-throughput SRAM closest to the CPU; high-capacity DRAM below the caches; specialized MRAM/RRAM where their non-volatile characteristics are useful; and NAND Flash at the storage level. This minimizes average memory-access latency while maintaining high overall system throughput and reasonable cost per bit.